

Intermediate (build the gate) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `release_gate_practice.py` (Intermediate section): "test before you release" is a habit people forget under pressure. A gate turns it into a rule the code enforces: no passing score, no update to what's live. **If not:** the Build Task's `deploy()` would be the first gate you ever wrote, guarding Project 5's real prompts.

All examples below assume `versions.json` already has `v1` (score 0.6) and `v2` (score 1.0) saved, the same way the `version_log` exercise builds it, and reuse the `run_eval_suite` stub from that exercise.

Basic Version

Approach 1 — the direct way

Story: run the suite, and only write the new version into `live_version.txt` if the score clears the bar. **If not:** every new version would go live whether it passed or not.

```
# release_gate_practice.py – Intermediate section
import json
import logging
import os

logging.basicConfig(level=logging.INFO)

THRESHOLD = float(os.getenv("RELEASE_THRESHOLD", "0.9"))

def load_versions(path):
    with open(path) as f:
        return json.load(f)

def run_eval_suite(prompt_text):
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

def deploy(version_name, threshold):
    versions = load_versions("versions.json")
```

```

target = None
for v in versions:
    if v["version_name"] == version_name:
        target = v

result = run_eval_suite(target["prompt_text"])
score = result["score"]
if score >= threshold:
    with open("live_version.txt", "w") as f:
        f.write(version_name)
    logging.info(f"{version_name} deployed, score {score}")
else:
    logging.warning(
        f"{version_name} NOT deployed, score {score} "
        f"below threshold {threshold}"
    )

deploy("v2", THRESHOLD)
deploy("v1", THRESHOLD)

```

Expected output (with no `RELEASE_THRESHOLD` set, so the default `0.9` is used):

```

INFO:root:v2 deployed, score 1.0
WARNING:root:v1 NOT deployed, score 0.6 below threshold 0.9

```

And `live_version.txt` on disk after both calls contains just `v2` — the rejected `v1` call never touched it. This version works correctly for what the exercise asks. It crashes with a confusing `TypeError: 'NoneType' object is not subscriptable` if `version_name` doesn't exist in the log at all (`target` stays `None`) — that's what Intermediate fixes.

Revision from Doc01 (Basic): two basic habits come back. First, the threshold is read with `os.getenv("RELEASE_THRESHOLD", "0.9")` instead of typing `0.9` into every call — the bar is a setting, not logic, so it belongs outside the code. Second, the two outcomes are logged at two different levels: a release that goes live is normal progress (`INFO`); a release the gate refuses is unusual but safe — the old version simply stays live — which is Doc01's definition of `WARNING` .

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a not-found guard, and a way to read the pointer back

Story: a missing version should be a clear message, not a confusing crash, and the release bar is a team decision that shouldn't silently fall back to a default. **If not:** a typo in a version name would crash with `NoneType`, and a lost `.env` line could quietly lower the bar.

```
# release_gate_practice.py – Intermediate section
import json
import logging
import os
from dotenv import load_dotenv

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)
logger.addHandler(logging.StreamHandler())

class MissingThresholdError(Exception):
    pass

def load_threshold() -> float:
    load_dotenv()
    raw = os.getenv("RELEASE_THRESHOLD")
    if raw is None or raw == "":
        # why: the release bar is a team decision – never guess it
        raise MissingThresholdError(
            "RELEASE_THRESHOLD is not set in .env – "
            "the gate will not guess its own bar"
        )
    return float(raw)

def load_versions(path: str) -> list[dict]:
    with open(path) as f:
        return json.load(f)

def find_version(versions: list[dict], name: str) -> dict | None:
    for v in versions:
        if v["version_name"] == name:
            return v
    return None
```

```

def run_eval_suite(prompt_text: str) -> dict:
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

def deploy(version_name: str, threshold: float) -> None:
    versions = load_versions("versions.json")
    target = find_version(versions, version_name)
    if target is None:
        logger.error(f"No such version: {version_name}")
        return

    result = run_eval_suite(target["prompt_text"])
    if result["score"] >= threshold:
        with open("live_version.txt", "w") as f:
            f.write(version_name)
        score = result["score"]
        logger.info(f"{version_name} deployed – score {score} >= {threshold}")
    else:
        score = result["score"]
        logger.warning(
            f"{version_name} NOT deployed – score {score} < {threshold}"
        )

def get_live_version() -> str | None:
    try:
        with open("live_version.txt") as f:
            return f.read().strip()
    except FileNotFoundError:
        return None

threshold = load_threshold()
deploy("v2", threshold)
deploy("v1", threshold)
deploy("v9", threshold)
print("currently live:", get_live_version())

```

Expected output, with `RELEASE_THRESHOLD=0.9` in `.env`:

```

v2 deployed – score 1.0 >= 0.9
v1 NOT deployed – score 0.6 < 0.9
No such version: v9
currently live: v2

```

(the first three lines are log lines — this handler has no formatter, so only the message shows)

And with `RELEASE_THRESHOLD` missing from `.env`, nothing is deployed at all. The last line of the traceback:

```
__main__.MissingThresholdError: RELEASE_THRESHOLD is not set in .env – the gate
will not guess its own bar
```

Revision from Doc01 (Intermediate): two habits come back. First, `load_dotenv()` plus Doc01's "required value is missing → raise a named error" rule. Basic used a default of `0.9`; that is fine for a practice script, but the release bar is a real team decision. If `.env` loses that line, a quiet default could let a weaker version go live without anyone choosing that. So the gate refuses to start. Second, a named logger with each outcome at its own level: `INFO` for a deploy, `WARNING` for a refused deploy (safe — nothing changed), and `ERROR` for `v9`, because that one operation failed completely, while the program itself can go on.

Difference from Basic: Approach 1 splits the lookup into its own `find_version` helper with a real not-found path (`if target is None: ... return`) instead of letting a missing version crash on `target["prompt_text"]`, adds full type hints, makes `RELEASE_THRESHOLD` a required `.env` value instead of an optional one with a default, logs through a named logger, and adds `get_live_version()` so the pointer can be read back safely (handling the case where `live_version.txt` doesn't exist yet). The score comparison and file-write logic are otherwise the same as Basic.

Approach 2 — a typed `DeployResult`, and a raised not-found error

Story: other code — a rollback script, a CI job, the Build Task's tests — needs to *read* the gate's decision, not parse printed text. A small dataclass gives `result.passed` and `result.score`, and a missing version raises instead of quietly returning. **If not:** the Build Task's `deploy_gate.py` would have no result a caller can check, and a missing version could be mistaken for "deploy refused".

This builds on Approach 1's `load_threshold`, `load_versions`, `find_version`, `run_eval_suite` and `logger` in the same section, and adds:

```
# release_gate_practice.py – Intermediate section
from dataclasses import dataclass

class VersionNotFoundError(Exception):
    pass

# why: the gate's decision as real fields a caller can check
@dataclass
class DeployResult:
```

```

version_name: str
passed: bool
score: float

def deploy_with_result(version_name: str, threshold: float) -> DeployResult:
    versions = load_versions("versions.json")
    target = find_version(versions, version_name)
    if target is None:
        # why: a missing version is an error, not "deploy refused"
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    score = run_eval_suite(target["prompt_text"])["score"]
    passed = score >= threshold
    if passed:
        with open("live_version.txt", "w") as f:
            f.write(version_name)
        logger.info(f"Deploy {version_name}: score {score}, now live")
    else:
        logger.warning(f"Deploy {version_name} refused: score {score}")
    return DeployResult(version_name=version_name, passed=passed, score=score)

outcome = deploy_with_result("v2", threshold)
print(outcome)
print(outcome.passed)

try:
    deploy_with_result("v9", threshold)
except VersionNotFoundError as e:
    logger.error(f"Deploy failed: {e}")

```

Expected output, with `RELEASE_THRESHOLD=0.9` in `.env` :

```

Deploy v2: score 1.0, now live
DeployResult(version_name='v2', passed=True, score=1.0)
True
Deploy failed: No saved version named: v9

```

(the first and last lines are log lines — this handler has no formatter.)

Difference between Approach 1 and Approach 2: Approach 1 decides and logs, which is enough for a person watching the terminal. Approach 2 returns the decision as data (`DeployResult`) and turns a missing version into a raised `VersionNotFoundError` , so other code can act on the result — and can't mistake "no such version" for "refused".

Which one should you actually write? Approach 2 — `DeployResult` is named in the Build Task's own requirements, and its `deploy_gate.py` returns exactly this kind of object. Approach 1 is the right first step to get the gate working at all.